23 AUGUST SESSION

Session Q&A: Environment Setup

You asked 212 questions during the session. Many of them were the same question wearing different clothes, so I have merged them into 48 and answered each one properly — with the commands, the numbers and the reasoning behind them.

QUESTIONS ASKED	ANSWERED HERE	STARTS WITH	ENDS WITH
212, from 130 of you	48 merged questions	Hardware — the biggest worry	What to do this week

Before you read on

Thank you for the volume of questions. It told me two things. First, most of you are further along than you think — you were asking about dependency resolution and VRAM budgets, not about what a terminal is. Second, a lot of you are anxious about hardware, and that anxiety is mostly unnecessary. I want to settle it on the first page.

> **You almost certainly do not need to buy anything.** If your laptop runs a browser and a code editor without complaint, it will carry you through this course. The heavy work — the large models — happens on someone else's machine, reached over an API or on a free cloud notebook. Running a model on your own laptop is a thing we do once, deliberately, with a small model, so that you understand what it costs. It is a lesson, not a requirement.

A second thing worth saying up front. Several of you wrote that the session moved quickly and assumed familiarity with the vocabulary. That is fair feedback and I have taken it. This document is written the other way round: it assumes you have not met these terms before, and it defines them where they appear. Where a question was really about course logistics — GPU credits, portal access, deadlines — I have grouped those at the very end, because those are decisions your programme coordinator owns rather than me.

And one thing that should reframe several of the hardware answers: **the labs and assignments are built around API calls.** That is where the assessed work sits, and where the employable skill sits. Running models on your own machine is an optional experiment we do so that you understand what an API is doing on your behalf — worth doing, never the thing you are graded on. Docker likewise appears only where a lab explicitly calls for it.

How the answers are organised

SECTION	COVERS	QUESTIONS
1	Your machine — RAM, VRAM, GPU, buying advice	Q1–Q9
2	Python versions and multiple installs	Q10–Q17
3	Virtual environments and package managers	Q18–Q25
4	Editors, notebooks and Colab	Q26–Q30
5	Running models locally versus in the cloud	Q31–Q34
6	Git, GitHub and keeping secrets out of them	Q35–Q37
7	Docker and shipping	Q38–Q40
8	Language choice, prerequisites and model concepts	Q41–Q46
9	The setup documents and how to use them	Q47–Q48
10	Questions for your coordinator, and free credentials	Closing

The stack, settled

A dozen questions asked some version of "which tool should I install?". Here is the whole list, and nothing on it is a surprise.

TOOL	STATUS	NOTE
Python 3.11.x	Required	Any patch level. 3.11.9 is fine, so is 3.11.4.
`venv` + `pip`	Default	What every setup document uses, so it is what I can walk you through line by line.
VS Code	Recommended	What I will be sharing my screen from. Not compulsory.
Git + a GitHub account	Required	Free account. You will push your capstone here.
Docker Desktop	For labs	Only where a lab or assignment calls for it. Not needed to write code or call an API.
Ollama	Optional	How you run a small model locally, for your own experimentation. Free, no key, one install.
Google Colab	Optional	Your fallback when your laptop is not enough. Free tier suffices.
Jupyter	Optional	Nice to have for experiments. Nothing in the course requires it.
Anaconda, `uv`, Poetry	Your choice	All fine. If you already work in one, keep using it — see Q20.

SECTION ONE

Your machine

Roughly a quarter of the questions were about hardware, and nearly all of them carried the same worry underneath: *am I going to be locked out of this course by my laptop?* The answer is no. Read Q1 and Q2 and then stop worrying about it.

Q1 I have 8 GB of RAM. The document says 16 GB. Am I in trouble?

Asked by Hanamantha B, Anindya Basak, Mohini Pathak, and three anonymous attendees

No. You can do this entire course on 8 GB. What you cannot do comfortably on 8 GB is run a language model on your own machine — and that is one afternoon of the course, not the course.

Here is the honest split. Every part of the course that involves writing code, calling an API, building a service and shipping it in a container is text and network work. It uses a few hundred megabytes. Your browser is heavier than your program will be. The one activity with a real appetite is loading a model's weights into memory, and even there a small model such as Qwen2.5-1.5B needs roughly 2 GB — which fits in 8 GB alongside your editor, if you close a few browser tabs first.

What will hurt on 8 GB is a 7-billion-parameter model, which wants about 5 GB just for weights and more for working memory. When we get there, you use Colab's free tier instead and lose nothing but a little convenience.

WHERE PEOPLE GET STUCK

The failure on a small-memory machine is rarely an error message. It is the laptop going quiet and unresponsive for two minutes while the operating system swaps memory to disk. If that happens, you have not broken anything — quit the process, and move that particular exercise to Colab.

Q2 Should I buy a new laptop for this course? Which model?

Asked by Arya Madhu, Manjula Sridhar, Devesh Kumar, Prasad Tendulkar, Yudhisthir Singh, Sumit Madaan, Amit Patil, Sachin Shinde and two anonymous attendees

Please do not buy a laptop for this course. I mean that plainly. Several of you described machines — an M3 with 16 GB, a 16 GB Mac, an RTX 4050 — that are more than sufficient, and asked whether to upgrade anyway. You do not need to.

If you are buying regardless, for reasons of your own, here is how the tiers actually break down.

TIER	SPECIFICATION	WHAT IT GETS YOU
Minimum	Any machine from the last six years, 8 GB RAM, 256 GB free-ish disk	Every exercise, every assignment, the capstone. Local models limited to the small ones; Colab covers the rest.
Comfortable	16 GB RAM. Apple M-series, or any recent Intel/AMD laptop	The above, plus local models up to about 7 B parameters without thinking about it. This is the sweet spot.
Generous	32 GB RAM and a discrete NVIDIA GPU with 8 GB+ VRAM (RTX 4060/4070/5070 class)	Local fine-tuning experiments and larger models at speed. Genuinely useful later, unnecessary now.

Laptop tiers. The course is designed against the minimum column.

Mac versus Windows: it does not matter. Apple Silicon has unified memory, which means the whole 16 GB is available to the model rather than a separate small pool — that is a real advantage for local inference. A Windows machine with an NVIDIA card is faster at training and has a wider tooling ecosystem. Both are fully supported; we have written a separate setup document for each.

The one specification I would push back on is disk. 256 GB fills up quickly once you have a few model downloads and Docker images. If you are choosing between more RAM and more disk at the same price, take the RAM, but keep an eye on free space.

WHERE PEOPLE GET STUCK

Someone asked about renting instead of buying. That is the better instinct. An hour of a cloud GPU costs roughly what a coffee does, and you will use it for perhaps twenty hours across the whole course. Buying a five-thousand-rupee-a-month machine to avoid a two-thousand-rupee total bill is the wrong trade.

Q3 Can I use my company laptop? It has restricted access.

Asked by Sunil Kishanrao Rathod, Rasmi Ranjan Ray and an anonymous attendee

Usually yes, with two caveats. The setup documents deliberately include a "no admin" path at every step where one is possible — Python, Git and VS Code can all be installed to your user folder without an administrator password, and the documents tell you how.

The two things that genuinely need administrator rights are Docker Desktop and, on Windows, enabling WSL2. If your IT policy blocks those, you have three options in order of preference: ask IT for an exception citing a training programme, use your personal machine for those two phases only, or use a cloud environment for the container work. None of these is a disaster.

The other caveat is the corporate proxy. If `pip install` hangs forever on a company network, that is almost always a proxy or certificate interception, not a broken Python. Your IT team will know the proxy address; `pip` accepts it through the `HTTPS_PROXY` environment variable.

One person asked whether to install into VirtualBox instead. You can, but I would not. A virtual machine adds a layer of confusion for a beginner — file sharing, networking, memory allocation — for no benefit here. Install natively.

One clarification on Docker, because it worries people more than it should: **Docker is not required for the bulk of the course.** Writing Python, calling an LLM API, building a chat script, running FastAPI — none of that touches Docker. You need it only where a lab exercise or an assignment explicitly asks for it, and that comes late. So if your IT team is slow to approve it, carry on with everything else and sort Docker out when a lab actually calls for it. It is not a blocker for getting started.

Q4 I do not have an NVIDIA GPU. How do I proceed? How do I check what I have?

Asked by Aryan Pandey, Shikha Kachroo, Anumodh Sharma

You proceed exactly as everyone else does. Nothing in the taught material requires a local GPU.

To find out what you have:

```
# Windows or Linux, if you have an NVIDIA card:
nvidia-smi

# macOS — Apple Silicon has an integrated GPU, always:
system_profiler SPDisplaysDataType | head -20

# Linux, any vendor:
lspci | grep -i vga
```

If `nvidia-smi` prints a table, you have an NVIDIA GPU and the number after the slash in the memory column is your VRAM. One of you shared exactly this output showing an RTX 4050 with 6141 MiB — that is a 6 GB card, and it will run small and mid-sized models happily.

If the command is not found, you do not have an NVIDIA GPU, and your models will run on the CPU instead. Slower, entirely functional. A 1.5-billion-parameter model on a modern CPU produces text at roughly reading speed, which is fine for learning.

Q5 What is the difference between RAM and VRAM? And how does VRAM differ from CPU cache?

Asked by Srilatha Etukuri, Ankit Kumar, Brajabandhu Mishra

Think of three desks of different sizes, each closer to the worker than the last.

	RAM	VRAM	CPU CACHE
Belongs to	The CPU	The GPU	Inside the CPU itself
Typical size	8–64 GB	4–24 GB on a laptop	8–64 MB
Speed	Fast	Roughly 10× faster	Roughly 100× faster than RAM
You control it?	Yes — you choose what to load	Yes — you choose what to load	No. The processor manages it invisibly.
Holds	Your program, your data, the operating system	Model weights, and the intermediate numbers during inference	Whatever the CPU touched a moment ago

The three memories. Only the first two are yours to plan around.

Why the distinction matters for us: a language model is a very large pile of numbers that must be multiplied against your input, over and over, for every single token it produces. A GPU can do thousands of those multiplications simultaneously, but only against numbers sitting in its own VRAM. If the weights do not fit in VRAM, they sit in ordinary RAM and get shuttled across, and the shuttling is the bottleneck — which is why a model that half-fits runs dramatically slower than one that fits entirely.

CPU cache is a different category. It is automatic, invisible, and far too small to hold model weights. You will never size a project around it.

Apple Silicon is the interesting exception. Its memory is *unified* — one pool shared by CPU and GPU, with no copying between them. On a 16 GB M-series Mac, roughly 11 GB of that is usable by a model. This is why modest Macs punch above their weight for local inference.

WHERE PEOPLE GET STUCK

One question asked how data is shared between system RAM and VRAM. On a discrete GPU it is an explicit copy across the PCIe bus, and it is slow enough that frameworks work hard to avoid it. On Apple Silicon there is no copy at all. This is the single biggest architectural difference between the two platforms.

Q6 How much memory does a model actually need? How do I size a GPU for a project?

Asked by Anumodh Sharma, Rahul Sengupta, Neeladree Chakravorty, Gajanana Garuda, Akhil Kumar, Manikandan and an anonymous attendee

This is the most useful piece of arithmetic in the whole session, so I will do it slowly.

A model is a list of numbers called weights. The count of those numbers is the "B" in the model's name — 7B means seven billion of them. Each number occupies a fixed amount of memory depending on its precision:

- **FP16 / BF16** (half precision) — 2 bytes per weight. The usual default.
- **INT8** (8-bit quantised) — 1 byte per weight. Slight quality loss.
- **INT4** (4-bit quantised) — 0.5 bytes per weight. Noticeable but often acceptable loss.

So the base calculation is simply:

```
weights memory  =  parameters  ×  bytes per parameter

7B model at FP16  =  7,000,000,000 × 2  =  14 GB
```

Then someone asked the sharp follow-up: why does a 14 GB model need ~16 GB of VRAM? Because the weights are not the only thing in memory. You also need room for the activations (the intermediate numbers as data flows through the layers), the KV cache (the model's memory of the conversation so far, which grows with context length), and the framework's own overhead. A rule of thumb that has served me well:

> **Budget 1.2× the weight size for inference, and 3–4× for fine-tuning.** Someone asked whether 1.5×, 2× or 20× are safe factors — 1.2× is the realistic inference number, and 20× is the right order of magnitude for full training from scratch, which we will not be doing.

MODEL	FP16	INT8	INT4	RUNS ON
1.5B	3.0 GB	1.5 GB	0.8 GB	Anything. This is our course model.
3B	6.0 GB	3.0 GB	1.5 GB	8 GB RAM machine, quantised
7B	14 GB	7.0 GB	3.5 GB	16 GB Mac, or 8 GB VRAM at INT4
13B	26 GB	13 GB	6.5 GB	24 GB card, or Colab
70B	140 GB	70 GB	35 GB	Multiple datacentre GPUs. Use an API.

Weight memory by size and precision. Add 20% for inference overhead.

Working backwards from a business requirement, as one of you asked: you do not start from parameter count. You start from the task, pick the smallest model that performs it acceptably in a test, measure its memory, then add the buffer. Choosing a model before testing is how projects end up paying for a 70B when a 7B would have done.

WHERE PEOPLE GET STUCK

The KV cache is the part people forget. It scales with how much conversation history you keep. A model that runs fine on short prompts can exhaust memory forty turns into a long chat. If you see out-of-memory errors only after sustained use, that is the cache, not the weights.

Q7 CUDA only works with NVIDIA. What about AMD, Intel, or Apple?

Asked by Gajanana Garuda, Sanidhya Pal

Every vendor has an equivalent, and the frameworks we use hide the difference behind one line of code.

HARDWARE	FRAMEWORK	IN PRACTICE
NVIDIA	CUDA	The default everywhere. Best library support.
Apple Silicon	Metal / MPS, and MLX	PyTorch supports MPS well. MLX is Apple's own framework, genuinely fast on M-series, worth knowing.
AMD	ROCm	Works, improving quickly, occasionally rough on consumer cards.
Intel Arc	oneAPI / IPEX	Functional, smallest ecosystem of the four.
No GPU	CPU	Always works. Slow but never blocked.

In PyTorch the whole thing collapses to this, and you should write it this way from the start so your code runs on any of our machines:

```
import torch

device = (
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)
print(f"Using {device}")
```

On Apple MLX specifically: it is excellent, and if you are on an M-series Mac and want to go deeper into local inference, it is the most rewarding thing to explore. It is not part of the course syllabus.

Q8 I have a workstation / large server. Should I use it directly or run Linux in a VM?

Asked by Ankit Jain, Gajanana Garuda

Directly, on the metal, every time. A virtual machine costs you GPU passthrough (fiddly to configure and easy to get wrong), some memory to the host, and a layer of debugging you do not need.

For the HP Z8 described — dual Xeon, 76 threads, 512 GB RAM, 8 GB GPU — install Ubuntu natively and you have a genuinely capable machine. Note that the constraint there is the 8 GB GPU, not the 512 GB of RAM.

And on the related question: yes, 128 GB of system RAM will run large models on the CPU. It will be slow — perhaps a token per second for a 70B model — but it works, and it is an excellent environment for experimenting with models you could never fit on a consumer GPU. Use quantised versions and be patient.

Q9 Do I need to pay for a GPU? What are the options, and what does "cost" include?

Asked by Prasad Tendulkar, Hema Jalmoru, Kesani Saithejaswini, Bikash Ranjan Sahu, Omprakash Panigrahi, Gajanana Garuda

Nothing in the taught course requires a paid GPU. If you choose to rent one for your capstone, here is the decision I would walk through.

WHERE SHOULD THIS WORKLOAD RUN?

Calling an API — OpenAI, Gemini, Claude	**Your laptop.** No GPU involved at all. Most of the course.
A small model, under 3B, to understand local inference	**Your laptop, via Ollama.** Free. Works on 8 GB.
A 7B–13B model, or a short fine-tuning run	**Colab free tier (T4, 16 GB).** Or Kaggle, which gives 30 GPU hours a week.
Sessions longer than the free tier allows, or bigger cards	**Colab Pro,** or a rented A100 by the hour.
Something that must run continuously and serve users	**A cloud VM with a GPU,** and now you must think about cost properly.

On what "cost" means, which was a good question: for rented cloud GPUs it is per-hour billing for the instance, whether or not you are using it — an idle GPU VM bills exactly like a busy one. Add storage for your model files, and network egress if you move data out. The classic mistake is leaving an instance running over a weekend. Set a budget alert on day one, before you launch anything.

For a bought GPU the cost is the card, plus electricity, plus the fact that it depreciates faster than almost any other hardware you own. For twenty hours of use across a course, renting wins decisively.

SECTION TWO

Python and its versions

Q10 Which Python version? 3.9, 3.11, 3.13, 3.14? Any particular patch level?

Asked by Vijet Shetti, Deepak Joshi, Brajabandhu Mishra, Sneha Jain, Pallishree Moharana and two anonymous attendees

3.11.x. Any patch level — 3.11.9, 3.11.7, 3.11.4 all behave identically for our purposes. Do not spend time hunting a specific one.

The reason is not that 3.11 is better. It is that the machine-learning ecosystem lags the Python release cycle by a year or more. PyTorch, transformers, and the various vector-database and orchestration libraries publish pre-built binary packages for the versions most people use. When you install on 3.14, `pip` often cannot find a matching pre-built wheel, falls back to compiling from source, and either takes forty minutes or fails with a wall of C++ errors. That is not a puzzle worth solving during a course.

3.9 is the other end of the same problem — some newer libraries have already dropped it. 3.11 sits in the middle where everything works.

WHERE PEOPLE GET STUCK

"Wheel" is worth knowing as a word. A wheel is a pre-compiled package, ready to install in seconds. When you see `pip` printing "Building wheel for..." and hanging, it means no pre-built wheel existed for your Python version and platform, and it is compiling. Nine times out of ten the fix is to be on 3.11.

Q11 I already have 3.13 / 3.14. Do I have to downgrade? Can two versions coexist?

Asked by Pallishree Moharana, Dinesh Landge, Vijet Shetti and three anonymous attendees

You do not downgrade, and you do not uninstall. You **add** 3.11 alongside what you have. Multiple Python versions coexisting on one machine is normal and expected — this is precisely the problem virtual environments were invented to solve.

Install 3.11 by whichever route your setup document specifies, and then be explicit about which one you are using when you create an environment:

```
# macOS / Linux — name the version explicitly
python3.11 -m venv .venv
source .venv/bin/activate
python --version        # now says 3.11.x

# Windows — the launcher picks the version for you
py -3.11 -m venv .venv
.venv\Scripts\activate
python --version        # now says 3.11.x
```

Once that environment is active, `python` means 3.11 inside it and your system Python is untouched outside it. Your other projects on 3.13 keep working. This is the whole point.

Q12 Should I uninstall the Python I already have?

Asked by Dinesh Landge and an anonymous attendee

No — and on macOS and Linux, actively do not. Your operating system uses its own Python for internal tooling. Removing it breaks parts of the system in ways that are unpleasant to repair. The Python at `/usr/bin/python3` on a Mac belongs to Apple, not to you.

On Windows there is no system Python, so removing an old one is harmless, but it is still unnecessary. Leave it. Virtual environments make version coexistence a non-issue, and cleaning up is a distraction from getting set up.

Q13 What is the difference between `python` and `python3`? My Mac only accepts `python3` and `pip3`.

Asked by Sanidhya Pal, Toshant Kamble

History. When Python 3 arrived, `python` still meant Python 2 on most systems, so `python3` was introduced to be unambiguous. Python 2 has been dead since 2020, but macOS and many Linux distributions kept the naming.

The practical answer is that this problem disappears the moment you activate a virtual environment. Inside an active venv, `python` and `pip` both exist and both point at your 3.11. So when a document writes `python script.py` and your shell says "command not found", the real message is *you have not activated your environment*.

```
$ python --version
command not found: python

$ source .venv/bin/activate
(.venv) $ python --version
Python 3.11.9                    # it works now
```

Outside an environment, mentally substitute `python3` and `pip3` and you will be fine.

WHERE PEOPLE GET STUCK

Look for `(.venv)` at the start of your prompt. Its presence or absence explains the majority of "command not found" and "module not found" errors that beginners hit. Before debugging anything else, check for it.

Q14 I installed 3.11 with Homebrew but `python3 --version` still says 3.9.6

Asked by an anonymous attendee; related question on Homebrew from Yashasvi Bhatnagar

Nothing has gone wrong. The install succeeded; your shell is just finding the old one first.

Your shell searches a list of folders, in order, called the PATH. The first matching program wins. Apple's Python lives in `/usr/bin`, Homebrew's in `/opt/homebrew/bin`, and if `/usr/bin` comes first in your PATH, Apple's wins. Diagnose it:

```
$ which -a python3
/usr/bin/python3                # found first — this is Apple's 3.9
/opt/homebrew/bin/python3       # yours, further down the list
```

You have two fixes. The clean one for this course: ignore PATH order entirely and call the version by name when creating your environment, as in Q11 — `python3.11 -m venv .venv`. Once inside the venv, PATH order is irrelevant.

The permanent one: put Homebrew ahead of the system directories by adding this line to `~/.zshrc`, then opening a new terminal.

```
export PATH="/opt/homebrew/bin:$PATH"
```

On Homebrew more broadly, since it was asked directly: it is a good tool and there is no reason to abandon it after six years. Its one genuine annoyance is that `brew upgrade` can move a Python version underneath a virtual environment that was built against it, which breaks the venv. The repair is to delete `.venv` and recreate it, which takes thirty seconds. That is the whole of the complaint.

Q15 Windows says Python is not installed, but I installed it

Asked by Hema Jalmoru

Two causes, and both are common enough that I would check them in this order.

First, the installer's checkbox. The Python installer has a box marked "Add python.exe to PATH" on its very first screen, unticked by default. If you missed it, Windows genuinely cannot find Python from the command line. Re-run the installer, choose Modify, and tick it.

Second, and stranger: typing `python` on a fresh Windows install opens the Microsoft Store. That is a placeholder Microsoft ships, not a real Python. Turn it off under Settings → Apps → Advanced app settings → App execution aliases, where you switch off both `python.exe` and `python3.exe`.

Then verify with the launcher, which is more reliable than `python` on Windows:

```
py --list          # every Python Windows knows about
py -3.11 --version # the one we want
```

On the related question of where to install from: use python.org, not the Microsoft Store version.

Q16 On Windows, do I need WSL, or can I install directly?

Asked by Sudhir Nigam, Roshani Sharma

Our Windows setup document uses WSL2, and I would follow it. WSL2 gives you a real Ubuntu inside Windows, which means every command in every tutorial, blog post and error-message search result you encounter for the rest of your career will work verbatim. Without it, you are permanently translating Linux instructions into PowerShell.

It is also what Docker Desktop uses underneath on Windows regardless, so you are installing it either way.

Native Windows Python does work, and if WSL2 is blocked by your IT policy, the document covers that path too. But if you have the choice, take WSL2 — it removes an entire category of friction.

Q17 I have a Mac and no Python at all. Where do I start?

Asked by Subhajit Chatterjee

You have Python — every Mac ships with one — but it is Apple's and it is old. Install your own:

```
# 1. Install Homebrew (one command, from brew.sh)
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 2. Install Python 3.11
brew install python@3.11

# 3. Verify
python3.11 --version
```

That is the short version. The Apple Silicon setup document walks through it with the verification step after each command and the "no admin" alternative if you cannot install Homebrew. Follow that rather than these three lines.

SECTION THREE

Environments and packages

Q18 Do I really need a virtual environment for every project?

Asked by Rohit Singh, Jinia Konar

Yes. One per project, without exception, and it costs you two commands.

The reason becomes obvious the first time you skip it. Project A needs `transformers` 4.38. Project B needs 4.20 because a library it depends on has not caught up. Installed globally, one of these two projects is broken at all times, and installing for one silently breaks the other. With environments, they never meet.

What a virtual environment actually is: a folder. That is the entire trick. It contains its own copy of the Python interpreter and its own `site-packages` directory, and activating it simply puts that folder at the front of your PATH.

WHAT A PROJECT FOLDER LOOKS LIKE

```
llm-starter/
├── .venv/                    ← the environment. Never commit this.
│   ├── bin/                      (Scripts\ on Windows)
│   │   ├── python                → your 3.11 interpreter
│   │   ├── pip
│   │   └── activate              → the script that switches it on
│   └── lib/python3.11/site-packages/
│       ├── openai/               ← installed libraries live here,
│       ├── requests/                not in your system Python
│       └── ...
├── .env                      ← your API keys. Never commit this either.
├── .gitignore                ← the file that enforces both "nevers"
├── requirements.txt          ← the list that lets someone rebuild .venv
├── chat_api.py
└── chat_local.py
```

Notice which files you share and which you do not. `.venv` is disposable and machine-specific — it is regenerated from `requirements.txt` in one command. `.env` holds secrets and never leaves your machine. Everything else is the project.

Q19 If I end up with a hundred environments, what happens to my disk?

Asked by Hakshay Sundar

A genuinely good question, and the honest answer is that `venv` does not deduplicate at all. A hundred environments each containing PyTorch means a hundred copies of PyTorch, and PyTorch is around 2 GB. The arithmetic is not kind.

Three mitigations, in the order I would reach for them.

Delete environments rather than projects. A `.venv` is rebuildable in one command from `requirements.txt`, so it is safe to delete any environment you have not touched in months. This is the habit worth forming.

```
# find the large ones
du -sh */.venv | sort -h | tail -20

# delete one; rebuild later with:
#   python3.11 -m venv .venv && pip install -r requirements.txt
rm -rf old-project/.venv
```

Clear the pip cache, which holds downloaded packages and grows quietly: `pip cache purge`.

Use a tool that deduplicates, if this becomes a real constraint for you. `uv` maintains a single content-addressed store and hard-links packages into environments, so ten projects sharing PyTorch store it once. If you are managing many projects on a small disk, that is a legitimate reason to prefer it — see Q20.

Q20 conda, uv, Poetry, venv — which should I use for this course?

Asked by Partha Pratim Chail, Supratik Panja, Rohit Manav, Writabrata Roy, Novendu Chakraborty, Kanthi Pavuluri, Neeladree Chakravorty, Devmallya Bandyopadhyay

My position: **use whichever you already know.** They all solve the same problem and none of them will hold you back. If you are productive in conda today, switching to please a course document is a waste of your evening.

That said, the three setup documents are written entirely in `venv` + `pip`, which makes it the default here for one practical reason — it is the path I can walk you through command by command, and the one I can debug with you when something fails. It is also built into Python, so there is nothing to install.

TOOL	BEST AT	CONSIDER IT WHEN
`venv` + `pip`	Being already installed and universally understood	You are starting out, or you want the documented path.
`uv`	Speed and disk efficiency — 10–100× faster installs	You are impatient, or juggling many projects. Drop-in replacement: `uv venv`, `uv pip install`.
conda	Non-Python dependencies — CUDA toolkits, compilers	You need a specific CUDA build, or you already live in it.
Poetry	Dependency resolution and publishing packages	You are building a library others will install.

To the person who said conda feels vast and `uv` feels simple: I agree, and `uv` is where the ecosystem is heading. If you want to use it for the course, translate the documents like this — `uv venv --python 3.11` in place of `python3.11 -m venv .venv`, and `uv pip install` in place of `pip install`. Everything else is identical.

WHERE PEOPLE GET STUCK

The one rule that matters more than the choice: do not mix them inside a single project. A conda environment with `pip` packages layered on top is the most reliable way to produce a broken environment that nobody can diagnose. Pick one per project and stay in it.

Q21 Is Anaconda needed? Should I install Miniforge on Windows or Mac?

Asked by Ajay Kumar, Smita Bagchi, Hari Prasadh, Neeladree Chakravorty, Kanthi Pavuluri, Devmallya Bandyopadhyay

Not needed. Nothing in this course requires it.

What Anaconda is, since it was asked directly: a bundle of Python plus several hundred pre-installed scientific packages plus its own package manager (`conda`) plus a graphical launcher. It came from the scientific-computing world, where installing NumPy or SciPy used to mean compiling Fortran libraries by hand. Anaconda solved that, and for that audience it remains excellent. It is also a multi-gigabyte install of which you will use perhaps five percent.

Miniforge is the same `conda` engine without the bundled packages and without the commercial licensing questions that surround Anaconda's distribution in corporate settings. If you want conda, Miniforge is the better choice — on Apple Silicon and Windows alike.

To the question about installing every library through Miniforge on an M4 Mac, sometimes combined with pip: you can, and it works, but read the warning at the end of Q20 first. Choose one manager per environment.

Q22 Why run `pip install` after `pip freeze`? Aren't the packages already installed?

Asked by Hari Krishna Mandla

They are — on *your* machine. The two commands point in opposite directions and serve different people.

```
# You, today — write down what you have:
pip freeze > requirements.txt

# Your teammate tomorrow, on a fresh machine — rebuild it:
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

`freeze` reads your environment and writes a list. `install -r` reads that list and builds an environment. You run `freeze`; someone else — or you, on a new laptop, or the Docker build in the final phase — runs `install -r`.

The reason it must be exact versions rather than package names is reproducibility. `openai` means whatever is newest today; `openai==1.35.0` means the version you tested against. Six months from now those are very different programs.

Freezing is not a finish line

Rereading the session, I think I caused this confusion myself, so let me correct it. Creating `requirements.txt` is not the end of the process. It is a **snapshot of your environment at one moment**. The moment you install another package or upgrade one, that snapshot is out of date and you freeze again.

```
pip install openai python-dotenv
pip freeze > requirements.txt        # snapshot 1

# a week later, you need something else
pip install fastapi uvicorn
pip freeze > requirements.txt        # snapshot 2 — do not forget this line
```

Forgetting the second freeze is one of the most common causes of "it works on my machine". Your code imports `fastapi`, your requirements file has never heard of it, and the next person's build fails. Make re-freezing a reflex after any install.

And running `pip install` twice is harmless

Several of you were nervous about re-running an install. Do not be. If the package is already there at a satisfying version, `pip` does nothing at all and tells you so:

```
$ pip install openai
Requirement already satisfied: openai in ./.venv/lib/python3.11/site-packages
(1.35.0)
```

Nothing is downloaded, nothing is reinstalled, nothing is broken. The same is true of `pip install -r requirements.txt` on an environment that already matches it — it checks each line and moves on. This makes it a safe command to run whenever you are unsure whether your environment is current, which in practice is often.

Q23 How do I know which library version to downgrade when versions conflict? How do I avoid dependency hell?

Asked by Neeladree Chakravorty

Mostly by not creating it. Four habits, in order of how much they save you:

One environment per project. Most dependency hell is cross-project contamination, and this eliminates it entirely.

Install everything in one command, not one at a time. `pip` resolves the whole set together and can find a combination that satisfies all of them; installed sequentially, each install only knows about what came before and will happily break an earlier package.

```
# good — the resolver sees all three at once
pip install "openai==1.35.0" "python-dotenv==1.0.1" "requests==2.32.3"
```

Freeze once it works, and commit that file. A working `requirements.txt` is the artefact that lets you return to a good state.

When it does break, read the error. Modern `pip` tells you exactly what it could not satisfy:

```
ERROR: Cannot install openai==1.35.0 and httpx==0.23.0 because
these package versions have conflicting dependencies.

  openai 1.35.0 depends on httpx<1,>=0.23.0
  langchain 0.1.0 depends on httpx<0.25.0
```

That is not a puzzle — it is an instruction. The overlap is `httpx` between 0.23 and 0.25, so pin something in that window. Use `pip index versions httpx` to see what is available, and `pip show httpx` to see what currently depends on it.

And when you are truly stuck: delete `.venv`, recreate it, and install everything in one command. Because environments are disposable, starting over costs two minutes and is very often faster than untangling.

Q24 When I ship a new version to a client, must they create a new environment every time?

Asked by Dev Midya

No. They update the one they have:

```
source .venv/bin/activate
pip install -r requirements.txt --upgrade
```

A fresh environment is only warranted when the Python version itself changes, or when an upgrade has left the environment in a state nobody can explain. Both are rare, and both are cheap to fix precisely because the environment is disposable.

This is, incidentally, the problem Docker solves permanently — you ship the environment along with the code and the question stops arising. That is Q38.

Q25 Which libraries do I need? Matplotlib, pandas, PyTorch — what should be ready before the next session?

Asked by Hari Prasadh, Pranav, Gokul R, Gargi Dutta and an anonymous attendee

Gokul asked for this split into required, good-to-have and optional, which is exactly the right way to present it. Here it is.

PACKAGE	STATUS	WHY
`openai`	Required	The API client. Also speaks to any OpenAI-compatible endpoint, including local ones.
`python-dotenv`	Required	Loads your keys from `.env` so they never appear in your code.
`requests`	Required	Plain HTTP. You will use it constantly.
`fastapi`, `uvicorn`	Required later	How you turn a script into a service. Covered in the Build phase.
`ollama`	Optional	Only if you want to drive local models from Python rather than the terminal. The `openai` client already talks to Ollama — see Q33.
`pandas`	Good to have	Any work with tabular data.
`matplotlib`	Good to have	Charts. Useful for evaluation results, not needed for setup.
`transformers`, `torch`	Good to have	Only when we load models directly rather than through Ollama. Large download — do not install speculatively.
`jupyter`	Optional	See Q28.
`numpy`	Automatic	Arrives as a dependency of nearly everything. Never install it deliberately.

For the next session, the four required ones are all you need:

```
pip install openai python-dotenv requests fastapi uvicorn
pip freeze > requirements.txt
```

SECTION FOUR

Editors, notebooks and Colab

Q26 Is VS Code mandatory? Can I use PyCharm, or just the terminal?

Asked by Susovan Paul, Kulbir Singh, M Venkata Ramana Raju, Kesani Saithejaswini, Hari Prasadh

Not mandatory. Use what you are fast in.

I will be sharing my screen from VS Code, so following along is marginally easier if you are in it too. It is also free, light, and has the best remote and container tooling of any editor, which matters in the final phase. PyCharm is a better Python IDE in several respects — its refactoring and debugger are excellent — and if you own it, use it.

The terminal alone is enough, if that is your preference. Nothing in the course depends on an editor feature.

The one thing that matters, whichever you choose: your editor must be pointed at the project's virtual environment, not your system Python. In VS Code that is `Ctrl/Cmd + Shift + P` → "Python: Select Interpreter" → the one with `.venv` in its path. In PyCharm it is Settings → Project → Python Interpreter → Add → Existing environment. Getting this wrong produces "module not found" errors for packages you can see are installed, and it is the single most common setup failure I see.

Q27 Which VS Code extensions do I need?

Asked by Amit Patil

Four, and only four. Extensions are where editors go to become slow.

EXTENSION	PUBLISHER	GIVES YOU
Python	Microsoft	Interpreter selection, debugging, the basics. Pulls in Pylance.
Jupyter	Microsoft	Run notebooks inside VS Code. Only if you want notebooks.
Docker	Microsoft	See and manage images and containers without leaving the editor.
WSL	Microsoft	Windows only, and essential if you followed Q16.

Install nothing else until you feel an actual need. A linter and formatter are worth adding later — Ruff is the current best answer — but they are noise while you are still getting a first program to run.

Q28 Is Jupyter Notebook mandatory?

Asked by Lalita Nayal, Brajabandhu Mishra and an anonymous attendee

No. Optional, and genuinely useful — worth having, never required.

Notebooks are for exploration: run a few lines, look at the output, adjust, run again, with everything staying in memory between cells. That loop is excellent when you are inspecting data or trying a prompt twenty ways. It is poor for anything you intend to ship — notebooks hide execution order, resist version control, and cannot be run as a service.

My working split, offered as a habit rather than a rule: explore in a notebook, then move anything that survived into a `.py` file. Everything we build in the Build and Ship phases is `.py`, because you cannot put a notebook in a container and call it an application.

If you want it: `pip install jupyter`, then `jupyter lab`. Or install the VS Code Jupyter extension and open a `.ipynb` file directly in the editor, which I prefer.

Q29 When should I use local VS Code and when Colab? Will assignments be on Colab?

Asked by Jinia Konar, Deepti Gupta, Smita Bagchi, Subrata Chattopadhyay, Amit Shah, Sudeep Mathews, Kanthi Pavuluri, Devmallya Bandyopadhyay

Default to local. Reach for Colab when you need a GPU you do not have.

The reason for that default is that the course is teaching you to build and ship applications, and applications do not live in notebooks. The Ship phase — Docker, GitHub, a running service — has no Colab equivalent. If you do everything in Colab, you will arrive at the capstone having never set up the thing the capstone requires.

USE LOCAL VS CODE FOR	USE COLAB FOR
Anything you will keep or ship	Anything needing a GPU you do not own
The API exercises — no GPU needed	Models larger than your RAM allows
Git, Docker, FastAPI, the capstone	Quick experiments on a borrowed machine
Work you want to still exist tomorrow	Fine-tuning runs

To Smita, who finds Colab efficient and dislikes conda — those are two separate preferences and I agree with both halves. Nothing here asks you to use conda. Use Colab freely where it suits you; just do the shipping work locally.

On creating a virtual environment inside Colab: you do not. Colab hands you a fresh disposable machine each session, which is the same isolation a venv provides. Just `!pip install` what you need at the top of the notebook. Do remember that the machine and everything on it

disappears when the session ends — mount Google Drive or push to GitHub before you close the tab.

WHERE PEOPLE GET STUCK

Colab's free tier disconnects after a period of inactivity and caps your GPU hours. Losing an hour of fine-tuning to a disconnect is a rite of passage. Checkpoint to Drive as you go.

Q30 Is there a browser-based environment that has all of this ready?

Asked by Akhil Kumar

Several, and they are a reasonable fallback if your machine is genuinely blocked — a locked corporate laptop, for instance.

GitHub Codespaces is the closest fit: a full VS Code in the browser attached to a real Linux machine with Docker available, and the free tier includes 60 hours a month. Google Colab covers notebooks and GPUs. Replit is a lighter option for small projects.

I would still encourage the local setup if it is at all possible. Part of what this course teaches is the ability to make a machine ready for work, and that skill only develops on a machine you own.

SECTION FIVE

Running models: local or cloud

Q31 When you say "running models locally", do you mean on my laptop or on a cloud machine?

Asked by Brajabandhu Mishra, Rishabh Raj, Dipayan De

"Local" means the weights are on a machine you control and the inference happens there — as opposed to sending your text to OpenAI or Google and receiving an answer. That machine can be your laptop or a cloud VM you rent; the distinction that matters is who holds the weights, not where the box is.

Why it matters in practice: local means your data never leaves your control, there is no per-token cost, and it works offline — which is why regulated industries care about it. It also means you own the memory constraints, the speed, and the operations.

In this course you will do both, deliberately. The setup documents build the same chat program twice — once against the OpenAI API, once against a model running on your own machine through Ollama — so you can hold the two side by side and feel the difference in latency, in cost, and in output quality. That comparison is the point of the exercise.

Q32 Will we deploy our own LLM? Is Qwen the model we will use — should I get familiar with it?

Asked by Viswajeet Samal, Brajabandhu Mishra

Yes to deploying. The Ship phase has you package a working chat service into a container and run it, which is deployment in every meaningful sense.

On Qwen: the setup documents use **Qwen2.5-1.5B** for the local exercise, chosen for one reason — it is small enough to run on the weakest machine anyone reported in the session, and it is genuinely capable for its size. It is not a model you need to study in advance. Everything you learn running it transfers directly to any other model, because Ollama presents them all through the same interface:

```
ollama pull qwen2.5:1.5b     # about 1 GB
ollama run qwen2.5:1.5b      # chat with it in the terminal

# swap the name and everything else stays the same
ollama pull llama3.2:3b
ollama pull phi3:mini
```

Get it pulled before the next session and you will be ready. No further familiarity required.

To set expectations clearly, because I was loose about this in the session: **the labs will not ask you to deploy your own model.** Lab exercises and assignments are overwhelmingly API calls — that is the skill being assessed, and it is the skill you will use at work. Running a model locally is there for your own experimentation: to get comfortable with open-weight models, to see what they can do, and to discover that several of them are genuinely capable and cost nothing to run, within whatever your hardware allows. Treat it as the optional half of the course, and a rewarding one.

Q33 Can I avoid API costs entirely by downloading a model and running it on my CPU first?

Asked by Prasad Tendulkar, Hema Jalmoru, Bikash Ranjan Sahu, Kesani Saithejaswini and several anonymous attendees

Yes, and I recommend it as a first step. Get something running on your own machine, feel how it behaves, then spend money on an API once you know what you are buying. It is a good order to learn in.

One correction first, because the naming trips everybody up. **You cannot download Gemini, GPT-4 or Claude.** Those are hosted products; the weights are not published. What you *can* download are the open-weight models these labs release alongside them:

THE HOSTED PRODUCT	THE DOWNLOADABLE SIBLING	NOTE
Google Gemini	**Gemma 3** — 270M, 1B, 4B, 12B, 27B	The one to start with. The small sizes are built for exactly this.
OpenAI GPT	**gpt-oss** — 20B and 120B	Real, Apache-licensed, and too large for most laptops. See below.
Anthropic Claude	None	API only.
Alibaba Qwen	**Qwen2.5** — 0.5B upwards	What our setup documents use, at 1.5B.

So the workflow you want is real; just substitute the right names. `ollama pull gemma3:1b`, not `ollama pull gemini`.

What actually runs on a CPU

All of these run without a GPU. What changes is speed, and how much RAM you need free.

MODEL	DOWNLOAD	RAM TO HAVE FREE	ON A CPU
`gemma3:270m`	~250 MB	Under 1 GB	Fast anywhere. Runs on a phone. Good for classification and extraction, weak at conversation.
`gemma3:1b`	~800 MB	~2 GB	Comfortable on any machine, including 8 GB laptops. A sensible first download.
`qwen2.5:1.5b`	~1 GB	~2 GB	Our course model. Same story.
`gemma3:4b`	~3 GB	~4–8 GB	The quality sweet spot for most people. Roughly 8–15 tokens per second on CPU — readable, not snappy.
`gemma3:12b`	~8 GB	~9 GB+	Wants 16 GB total and real patience on CPU. Better with a GPU.
`gpt-oss:20b`	~13 GB	16–32 GB	OpenAI's own guidance is 16 GB of memory minimum. Not a realistic CPU-only target for most of you.

Figures assume 4-bit quantised builds, which is what Ollama pulls by default. Add your operating system's own usage on top.

On the CPU itself: any processor from the last five years or so will do. More cores help — Ollama uses them all — but memory is the constraint that decides whether a model runs at all, and memory bandwidth is what decides how fast. This is why an M-series Mac feels dramatically quicker than a similarly-priced Windows laptop at the same task.

The nice part: the code does not change

Ollama exposes an OpenAI-compatible endpoint, so the same script talks to either. This is the whole reason we teach it this way — experiment locally for free, then move to a paid API by editing two lines.

```
from openai import OpenAI

# Local — free, private, no key needed
client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
MODEL = "gemma3:1b"

# Paid API — same client, same call below
# client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
# MODEL = "gpt-4o-mini"

reply = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "Explain a virtual environment in two
sentences."}],
)
print(reply.choices[0].message.content)
```

Start local, get the program working, then flip the two commented lines when you want the stronger model. Bear in mind though that the labs are built around API calls, so do not spend so long in local mode that you arrive without a working key.

WHERE PEOPLE GET STUCK

A small local model is not a small version of GPT-4o. A 1B model will lose the thread of a long conversation, invent facts confidently, and struggle with anything multi-step. That is not a broken install — it is what a billion parameters buys. Use local models to learn the mechanics and to feel the difference; use the API when output quality is the point.

Q34 Why download a model with code? It is a one-time job — why not do it by hand?

Asked by Anumodh Sharma

You can, and for a single model on a single laptop it makes no difference. The reason we write it as code is that "one machine, one time" stops being true almost immediately.

The code path gets you: a Docker image that fetches its own model when it builds, so a colleague can run your project without a set of manual instructions; a version pinned in a file rather than remembered; automatic caching, so the second run does not re-download two gigabytes; and resumable transfers when a large download drops halfway.

It is the same principle as `requirements.txt`. Anything a human has to remember to do by hand is something that will eventually be forgotten, done differently, or done by someone who was not there when you explained it.

SECTION SIX

Git, GitHub and secrets

Q35 What am I meant to do with the repository you shared? And can I clone other repos to try?

Asked by Shefali Agarwal, Pallishree Moharana

The repository I shared is reference material — read it, clone it, run it if you like. It is not an assignment and there is nothing to submit from it.

Cloning other public repositories to explore is exactly the right instinct, and the pattern is always the same:

```
git clone https://github.com/<owner>/<repo>.git
cd <repo>
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Note the ordering: environment first, install second. Running `pip install -r requirements.txt` without an active environment installs someone else's dependency list into your system Python, which is precisely the mess Q18 exists to prevent.

Do expect other people's repositories to break. Public research code is frequently pinned to old library versions or an old Python. That is not your failure, and a repository that will not install is not worth an evening.

Q36 If `.env` is never pushed to Git, how does anyone else know which variables to set?

Asked by Ankit Kumar

This was the sharpest question of the session. The answer is a convention, and it is the exact analogue of `requirements.txt` that you were reaching for: a file called `.env.example` that lists the *names* with the *values* removed. It is committed; the real `.env` is not.

```
# .env.example  — committed to Git, safe to share
OPENAI_API_KEY=
HUGGINGFACE_TOKEN=
MODEL_NAME=gpt-4o-mini
OLLAMA_HOST=http://localhost:11434

# .env  — on your machine only, never committed
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxx
HUGGINGFACE_TOKEN=hf_xxxxxxxxxxxxxxxxxxxx
MODEL_NAME=gpt-4o-mini
OLLAMA_HOST=http://localhost:11434
```

Someone joining your project copies one to the other and fills in their own keys:

```
cp .env.example .env      # then edit .env with your own values
```

Two details that make it work. Your `.gitignore` must exclude the real file while keeping the example — `.env` on one line, `!.env.example` on the next. And it is worth having your program fail loudly at startup when a variable is missing, rather than mysteriously three functions later:

```
import os
from dotenv import load_dotenv

load_dotenv()

key = os.getenv("OPENAI_API_KEY")
if not key:
    raise SystemExit("OPENAI_API_KEY missing — copy .env.example to .env and fill
it in")
```

WHERE PEOPLE GET STUCK

If you ever do commit a key by accident: rotate it immediately, before anything else. Do not delete the commit and assume you are safe — Git history is recoverable and automated scanners find exposed keys within minutes. Revoke the old key at the provider, issue a new one, then clean the history at your leisure. The setup documents cover the cleanup in the Connect phase.

Q37 Once I build something in a virtual environment, can I publish it to GitHub or Hugging Face?

Asked by Jinia Konar

Yes, and you will. They serve different purposes and you will likely use both.

GitHub takes your source code — the `.py` files, `requirements.txt`, `Dockerfile`, `README`, `.env.example`. Never `.venv`, never `.env`. Someone clones it and rebuilds the environment from your requirements file.

Hugging Face Spaces takes a running application. You push similar files and Hugging Face builds and hosts it at a public URL, with a free CPU tier that is entirely adequate for an API-backed chat app. For a capstone you want someone to actually click on, this is the shortest path from code to a live link.

Both are covered in the Ship phase of the setup documents.

SECTION SEVEN

Docker and shipping

Q38 Why do we need Docker? What is an image?

Asked by Achint Maheshwari, Shivanshu Singh Bisht, Devmallya Bandyopadhyay

Docker exists to end the sentence "it works on my machine".

`requirements.txt` pins your Python packages, which is most of the problem but not all of it. It says nothing about your Python version, your operating system, the system libraries your packages compile against, or the environment variables you had set. Docker packages all of it — the operating system, the interpreter, the libraries, your code — into one artefact that runs identically on your laptop, a colleague's laptop, and a server.

IMAGE AND CONTAINER

IMAGE

A read-only template. Built once from your `Dockerfile`. Contains OS + Python 3.11 + your packages + your code.

The recipe, written down.

`docker run →`

CONTAINER

A running instance of that image. Start ten from one image; they do not interfere. Stops, and its changes vanish.

The meal, cooked.

The commands you will meet are few:

```
docker build -t my-chat-app .        # Dockerfile → image
docker run -p 8000:8000 my-chat-app  # image → running container
docker ps                            # what is running
docker images                        # what images exist
docker stop <container-id>           # stop one
```

The payoff arrives at the capstone. "Here is my GitHub link, run `docker run`" is a complete handover. Without it you are writing a page of setup instructions and answering questions about them for a week.

Q39 Does the Docker container need to run inside the virtual environment?

Asked by Brajabandhu Mishra

No — and this is a distinction worth getting straight, because it confuses almost everyone at first.

They solve the same problem at different scales. A virtual environment isolates Python packages within one operating system. A container isolates the entire operating system. The container *replaces* the venv; it does not sit inside one.

So the layering is: your laptop runs Docker, Docker runs a container, and inside that container is a clean Linux with Python 3.11 and your packages installed directly. No venv inside, because the container is already the isolation.

In practice you still use a venv on your laptop while developing, and Docker only when you package. The `requirements.txt` your venv produced is what the `Dockerfile` reads. They cooperate; they do not nest.

Q40 How is packaging an app as a Docker image different from deploying components to a cloud environment? How do I choose?

Asked by Vaidehi Kulkarni

They are not alternatives — they compose. Docker is how you *package*; cloud deployment is *where you run* the package. Almost every modern cloud service accepts a container image as its input.

The real choice is how much of the running you want to manage:

APPROACH	YOU MANAGE	CHOOSE WHEN
Container on a VM	The machine, updates, scaling, uptime	You need specific hardware, such as a particular GPU.
Managed container service (Cloud Run, App Runner, Container Apps)	Only the image	Almost always. This is the default for a service like ours.
Serverless functions	Only the function code	Short, bursty, event-driven work. Awkward for model loading.
Managed platform (HF Spaces, Streamlit Cloud)	Nothing	Demos and capstone projects. Fastest route to a public link.

To the related question about clouds that build the container for you: yes, several will take a repository and produce the image themselves. That is convenient, and I would still have you write a `Dockerfile` by hand once. The generated ones are opaque, and when a build fails at two in the morning you want to understand what it was doing.

SECTION EIGHT

Language, prerequisites and concepts

Q41 Why is it always Python? Why not R, C++, Java or JavaScript?

Asked by Dev Midya, Achint Maheshwari, Nandu Krishnan, Anumodh Sharma

Because of where the libraries are, and for very little else. Python won this by accident of history and then compounded the advantage.

The observation that C++ is faster than Python is correct, and it does not matter here — because the Python you write for machine learning is not doing the arithmetic. PyTorch's numerical core *is* C++ and CUDA. Your Python is a thin scripting layer that arranges the work and hands it to compiled code. You get C++ speed in the inner loop and Python's readability everywhere else.

LANGUAGE	WHERE IT IS GENUINELY STRONG	WHY NOT HERE
Python	Every ML framework, every model release, every tutorial	—
R	Statistics, econometrics, publication-quality plots	Almost no deep-learning ecosystem. Excellent tool, wrong domain.
C++	The inside of PyTorch; inference engines like llama.cpp	You would spend your time on memory management, not models.
Java	Enterprise services, large systems	Real libraries exist (DJL, LangChain4j) but lag Python badly.
JavaScript	Anything in a browser; the whole application layer	Fine for calling an API — see below. Not for training.

On JavaScript specifically, since it was asked directly: if you are building a web application that calls an LLM API, JavaScript is a perfectly good choice and there are mature SDKs. It is not a good choice for training, fine-tuning, or working with weights. In this course we use Python because everything we do sits on that side of the line.

The practical point underneath all of this: the language is not the skill. What you are learning — how models behave, how to prompt them, how to evaluate outputs, how to ship a service — transfers to whatever language your team uses.

Q42 How much Python do I need? I am new to it — can I follow? Any tutorials?

Asked by Sharath Babu, Balwant Singh, Shivanshu Singh Bisht, Mahesh Agarwal

Less than you fear. You need to read Python confidently and write it slowly. Specifically: variables, lists and dictionaries, `if`, `for`, functions, importing a module, reading a file, and handling an error with `try/except`. That is a weekend, and it is enough.

What you do *not* need: classes and inheritance, decorators, async, metaclasses, comprehension gymnastics. They appear in code you read; you can look them up when they do.

To Balwant, who has C++, Objective-C and Swift: you have all the concepts and are only missing syntax. The official tutorial at `docs.python.org/3/tutorial/` is written for exactly your situation and will take you an afternoon. Skim it, then start writing — you will be productive immediately and will spend a month unlearning semicolons.

For genuine beginners, in order: the official tutorial for reference, "Automate the Boring Stuff with Python" (free online) for a gentler on-ramp, and then write something small that you actually want. Reading tutorials indefinitely is the trap. Write a twenty-line script that does something useless but yours.

And to the question of whether mastering AI requires mastering coding — no. It requires being comfortable enough with code to build, debug and reason about a system. That is a much lower bar than mastery, and you clear it by building things, not by studying first.

WHERE PEOPLE GET STUCK

Several of you apologised for asking basic questions. Please stop doing that. The questions in this session were good, and the person who asks "what is an image in Docker?" in front of four hundred people has done everyone else a favour.

Q43 If the weights are fixed and the input is identical, shouldn't the output be identical?

Asked by Balaji Rajadurai, Ankit Kumar

It would be, if the model returned its answer directly. It does not. At each step the model produces a probability distribution over the entire vocabulary — perhaps fifty thousand candidate next tokens, each with a score — and then something *samples* from that distribution. The sampling is where the variation lives.

Temperature controls the shape of that distribution before sampling. High temperature flattens it, giving unlikely tokens a real chance. Low temperature sharpens it toward the leader. At temperature 0 you are no longer sampling at all — you take the highest-probability token every time, which is called greedy decoding.

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Name a colour"}],
    temperature=0,      # greedy: always the most likely token
    seed=42,            # for the sampling, when temperature > 0
)
```

Now the follow-up, which was the better question: even at temperature 0, is it truly deterministic? In practice, no, and for three reasons. Floating-point addition is not associative, so GPU operations that run in parallel can produce results that differ in the last bits — occasionally enough to flip which of two near-tied tokens leads. Batching changes those groupings, and you do not control who else is batched with you on a shared endpoint. And, as Ankit noted, the provider may update the model behind the same name without telling you.

The lesson for building: never assume identical output. Design systems that tolerate variation — validate structure rather than comparing strings, and if you need reproducibility, pin the model version and run it yourself.

Q44 Please explain weights properly.

Asked by Hema Krishna Chaitanya Varikuti

A weight is a single number that says how strongly one thing influences another. That is all it is. A model with seven billion parameters is a structured arrangement of seven billion such numbers.

Start smaller. Suppose you predict a house price from its size, age and location. You might write:

```
price = (size × 4200) + (age × −1500) + (location_score × 90000) + 250000
```

The numbers 4200, −1500, 90000 and 250000 are weights. Nobody chose them by hand — they were found by showing the equation thousands of real houses and nudging each number in whichever direction reduced the error. That nudging process is training.

A language model is the same idea at an inconceivable scale and with a much richer structure. Instead of three inputs it has thousands of dimensions representing a token's meaning in context; instead of one equation it has dozens of layers, each feeding the next; instead of predicting a price it predicts a probability for every possible next token. But the mechanism is unchanged: multiply inputs by weights, add them up, pass the result on.

Which is why "downloading a model" means downloading a file of numbers, and why a 7B model at 2 bytes per number is a 14 GB file. Everything the model knows is encoded in the relative sizes of those numbers, and inference is simply running your text through them.

Q45 How is text turned into tokens, and how do I use fewer of them?

Asked by Brijesh Miglani, Sanju Gill and an anonymous attendee

A token is a chunk of text — usually a common word, a word fragment, or a piece of punctuation. Models work in tokens because a vocabulary of whole words would be enormous and would still miss anything unusual, while individual characters carry too little meaning. Fragments are the compromise.

Rough conversions worth memorising: about 4 characters per token in English, or 0.75 words per token. A thousand words is roughly 1,300 tokens. Non-English text costs considerably more per word, and code sits somewhere in between.

```
pip install tiktoken

import tiktoken
enc = tiktoken.encoding_for_model("gpt-4o-mini")
print(len(enc.encode("Your prompt here")))
```

On reducing token usage, in descending order of effect:

- **Do not resend the whole conversation every turn.** This is the largest cost in nearly every chat application, because each turn silently includes all previous ones. Summarise older turns, or keep only the last few.
- **Retrieve, do not stuff.** Sending an entire document so the model can answer one question about it is the expensive mistake. Retrieve the three relevant paragraphs instead. That is what RAG is for, and we will cover it.
- **Choose the smaller model when it suffices.** The price difference between tiers is often tenfold. Most classification and extraction work does not need the largest model.
- **Cap the output.** `max_tokens` costs nothing to set and prevents an unexpectedly long answer.
- **Use prompt caching** where the provider offers it — a long system prompt reused across requests can be billed at a fraction after the first call.

Sanju asked about parallelism in relation to token optimisation: they are separate concerns. Parallelism reduces wall-clock time by issuing several requests at once; it does not reduce the token count. Batch by all means, but the savings above are what move the bill.

Q46 Why is FastAPI faster than a normal Python API?

Asked by Gajanana Garuda

Two reasons, and the first is the one that matters for us.

It is asynchronous. A traditional synchronous framework handles one request per worker at a time — when your code is waiting on something, that worker sits idle. FastAPI, running on an async server, releases the worker during the wait and serves other requests, then resumes.

For an LLM application this is close to decisive, because your service spends nearly all of its time waiting on someone else's model. A request that takes two seconds might involve five milliseconds of your code and 1,995 milliseconds of waiting. Synchronously, one worker serves one user for two seconds. Asynchronously, that same worker serves hundreds.

```
@app.post("/chat")
async def chat(message: str):
    response = await client.chat.completions.create(...)   # yields while waiting
    return {"reply": response.choices[0].message.content}
```

It validates with compiled code. Type hints on your function are turned into Pydantic models whose validation core is written in Rust. You get request validation, clear error messages and automatic interactive API documentation, at negligible cost.

The honest caveat: the `async` keyword only helps if everything inside is also async. One synchronous blocking call in an async endpoint stalls the entire event loop and you end up slower than where you started. That trap is worth knowing before you meet it.

SECTION NINE

The documents, and how to use them

Q47 Which document do I read, and in what order?

Asked by Shreema Gupta, Kulbir Singh, Kapil Kumar Agrawal, Vijet Shetti, Elizabeth Nanda, Brajabandhu Mishra, Sai Keerthana Donthu, Champaknath Kadaba and several anonymous attendees

There are four documents. Read one, follow one, ignore the other two.

DOCUMENT	WHEN	WHAT IT DOES
AI Engineering Environment Setup — Field Manual	First, once	The shape of the whole thing — what you are installing and why each piece exists. Twenty minutes of reading. Do not skip it; the setup documents assume its vocabulary.
Setup — Apple Silicon	Follow line by line	M1/M2/M3/M4 Macs. Homebrew, `zsh`, unified memory.
Setup — Linux	Follow line by line	Ubuntu 22.04+ and Debian 12+. `apt`, `bash`.
Setup — Windows	Follow line by line	Windows 10/11 via WSL2, with a native fallback if WSL2 is blocked.

The three setup documents are complete and independent. They deliberately repeat each other rather than cross-reference, so you never have to hold two open at once. Pick yours and stay in it.

They all follow the same five phases, and each numbered step ends with a **Verify** command that proves it worked. Run it. The verification is not decoration — it is what stops a small mistake in step 4 from surfacing as an inexplicable failure in step 14.

THE FIVE PHASES, IN EVERY DOCUMENT

PREPARE	INSTALL	CONNECT	BUILD	SHIP
The terminal, PATH, and the habit of verifying every step	Python 3.11, a virtual environment, pip	Git, GitHub, API keys, and secrets kept out of both	The same chat program twice — via API, and locally	Into a container, and up to GitHub

Set aside about four unhurried hours. It is not four hours of work — it is two hours of work and two hours of downloads, so start it when you can leave it running.

WHERE PEOPLE GET STUCK

The commonest failure mode is skipping ahead because a step looks familiar. The documents are ordered by dependency, not by difficulty. If a Verify fails, stop there — the troubleshooting table at the end of that step is where to look, and nothing further down will work until it passes.

Q48 The three platforms use different commands. Is there one place to compare them?

Asked by Toshant Kamble, Kesani Saithejaswini, Gargi Dutta

Here. Everything else in your setup document is platform-specific prose; this is the translation table.

TASK	MACOS	LINUX	WINDOWS (POWERSHELL)
Check Python	`python3 --version`	`python3 --version`	`py --version`
List all Pythons	`ls /usr/bin/python*` `brew list | grep python`	`ls /usr/bin/python*`	`py --list`
Install Python	`brew install python@3.11`	`sudo apt install python3.11 python3.11-venv`	python.org installer
Create venv	`python3.11 -m venv .venv`	`python3.11 -m venv .venv`	`py -3.11 -m venv .venv`
Activate	`source .venv/bin/activate`	`source .venv/bin/activate`	`.venv\Scripts\Activate.ps1`
Deactivate	`deactivate`	`deactivate`	`deactivate`
Where is a program	`which -a python3`	`which -a python3`	`where.exe python`
Environment variable	`export KEY=value`	`export KEY=value`	`$env:KEY="value"`
Shell config file	`~/.zshrc`	`~/.bashrc`	`$PROFILE`
Check GPU	`system_profiler SPDisplaysDataType`	`nvidia-smi`	`nvidia-smi`
Delete a folder	`rm -rf .venv`	`rm -rf .venv`	`Remove-Item -Recurse .venv`

If you followed Q16 and installed WSL2, use the Linux column throughout.

One Windows-specific trap: if activation fails with a message about execution policies, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once and try again. This is normal and safe.

On the session itself

Several of you raised things about the session rather than the setup, and they deserve answers rather than silence.

The PDFs on the GitHub repository would not render. Reported by Krishna Karunamoorthy and Balwant Singh, and confirmed by others. GitHub's inline PDF viewer fails on larger files. Use the **Download raw file** button at the top right of the file view and open it locally — the file itself is fine.

The RAM-versus-VRAM slide was badly formatted on Windows. Noted by Karan Malik. Q5 in this document is that content, reformatted and expanded. Use it instead of the slide.

The session moved too fast and assumed the vocabulary. Raised by Kulbir Singh and echoed by Mahesh Agarwal. Fair, and taken. This document is my first correction — it defines terms where they appear rather than assuming them. I will slow the pace on the setup material.

A live hands-on setup walkthrough. Asked for by many of you — Devmallya Bandyopadhyay, Saswat Panda, Prajyot Mhalaskar, Yuvaraj Kumar and others. I have passed this on; scheduling is not mine to decide. In the meantime the setup documents are written to be followed without me.

Making the Q&A visible to everyone. Requested by Toshant Kamble. This document is the answer for this session — every question is here regardless of who asked it.

CLOSING

Not mine to answer, and what to do now

A group of questions concerned programme logistics rather than engineering — and I would be guessing if I answered them. **Please take these to your programme coordinator**, who has the actual answers.

QUESTION	ASKED BY
Will upGrad or IIT Kharagpur provide GPU access, cloud credits or a lab environment?	Devmallya Bandyopadhyay, Shubhra Bakshi, Sunil Kishanrao Rathod, Dipayan De, and others
Are API keys funded by the programme, or purchased by us?	Shreema Gupta, Anonymous
Will there be a live hands-on setup workshop, and when?	Devmallya Bandyopadhyay, Saswat Panda, Prajyot Mhalaskar, Yuvaraj Kumar
Where exactly are the resources on PRISM / LXP? Is PRISM the same as the LXP?	Elizabeth Nanda, Pragati Kapoor, Honey Saini, Sai Keerthana Donthu
By when must the setup be complete? Deadlines and the diagnostic test?	Satyendra Avadhani, Anonymous
How do we reach fellow learners and the IIT Kharagpur PhD scholars for the capstone?	Jinia Konar
Where do we get help when setup fails outside session hours?	Lokesh Priyadarshi, Abhinav Yatendra Jain, Anirban Mukherjee
When are materials released relative to sessions?	Nikesh Tripathi

Getting credentials without spending much

Whatever the programme provides, it is worth knowing how to obtain your own. Every route below is free or close to it.

SERVICE	COST	NOTES
Ollama	Free, always	No account, no key, no network. The most reliable thing on this list — start here.
Google AI Studio	Free tier	A Gemini API key in about two minutes with a Google account, with a generous free request allowance. The easiest paid-grade API to get started on.
Hugging Face	Free	Free account and access token. Needed to download many models; also gives a free Inference API and a free CPU tier on Spaces.
Google Colab	Free tier	A T4 GPU with 16 GB VRAM, with usage limits and idle disconnects. Pro is inexpensive if you need longer sessions.
Kaggle Notebooks	Free	Around 30 GPU hours a week — more generous than Colab's free tier and less known.
OpenAI	Pay as you go	Prepaid, roughly a $5 minimum. That balance goes a very long way on a small model — set a hard usage limit in the dashboard on day one.
GitHub Education	Free	If you have any student or academic affiliation, the Student Developer Pack includes Copilot and various cloud credits.
AWS / GCP / Azure	Free credits	New accounts carry trial credits, typically $200–300. Enough for the whole course if spent carefully. Set a budget alert before launching anything.

If you do only one thing from this table: get an Ollama install and a free Gemini key. Between them you can complete every exercise without spending a rupee.

What to do this week

01 **Do not buy hardware**

The machine you have is enough. Revisit this in six months if you still feel the need.

02 **Read *AI Engineering Environment Setup — Field Manual*, then your platform's setup document**

Twenty minutes, then about four hours. One document, top to bottom, running every Verify.

03 **Get one API key working**

A free Gemini key counts. The point is a program of yours that receives a reply from a model.

04 **Finish before the next session**

We build on a working environment. Arriving without one means spending the session catching up rather than learning.

If you get stuck, note the exact command and the exact error message before asking — that pair usually contains the answer, and it always makes the question answerable.

A word on where this comes from, and where to reach me

Everything in this document comes from industry practice — how these tools are actually used on working projects. That is its value and also its limitation. **The faculty at IIT Kharagpur own the syllabus, and where their guidance differs from mine, theirs governs.** They may narrow some of what I have described, set different requirements, or place topics out of scope for the lectures entirely — running LLMs locally is a likely candidate, since it sits outside what the labs assess. If you find a difference between this document and what you are told in a lecture, follow the lecture.

If you hit a problem with anything covered here, write to me at **amit.s.pandey@gmail.com** and I will reply. Please keep that address to questions arising from this session and the setup — it is the only way I can answer everyone.

For anything broader — career direction, where the field is heading, what to learn next — LinkedIn is the better place, and I would genuinely rather have those conversations there than lose them in an inbox. Do connect: linkedin.com/in/amitpandeyprofile

Amit

Amit Pandey · Instructor